

Introduction to Deep Learning

Virtsudo

May 17, 2026

1 Data Preprocessing

Initially the dataset had **31507** samples, however after necessary data cleaning steps (convert infinite numbers to NaN, removing NaNs and duplicate samples) it dropped to **29386** samples. We split the dataset to train (60%), validation (20%), and test (20%) sets, stratify set dataset labels to achieve balanced splitting across sets.

Before normalization, we plot the boxplot of features Fig. 1, to check presence of outliers. Since all features numeric, we used Scaler to normalize features. We discovered, that features *Flow Bytes/s*, and *Flow Duration* have large outliers, by result of this we decided to use **RobustScaler** instead of StandardScaler. As labels are class names, we used **LabelEncoder** to convert class names to numbers.

2 Shallow Neural Network

In this section we worked on correctly choosing Shallow Learning model's hyperparameters, affection of features and their change on model performance. In more detail, we discussed which hidden number of neurons is better, why and how to choose it, the same reports for activation and loss functions.

2.1 Three architectures

We trained three Shallow Neural Networks varying only the hidden neurons 32, 64, 128, with fixed hyperparameters: linear activation, batch size 64, cross-entropy loss, AdamW optimizer, lr=5e-4, 100 epochs with early stopping (patience=5, min_delta=1e-3).

During training, all models have converged by a steady decrease in training and validation losses. All three models finished by early stopping, to avoid overfitting, hidden=32 is finished at 24 epochs with the best validation = 0.296, while hidden=128 is completed at 16 epochs with the best validation loss = 0.301. The best validation loss = 0.286 (at 33 epochs) across models corresponds to hidden = 64. Despite having more capacity, hidden=128 converged faster but to a slightly worse minimum, suggesting it may have overfit the training signal early.

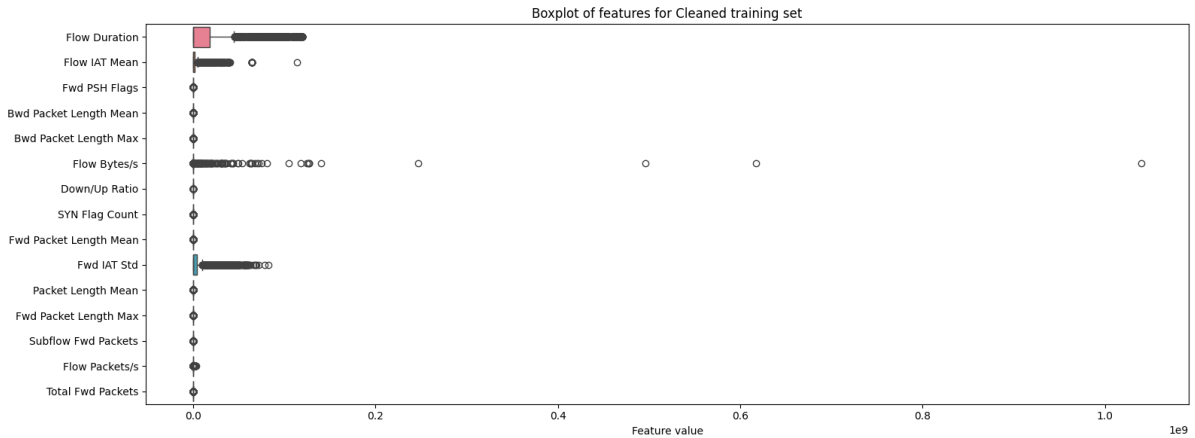


Figure 1: Boxplot of feature distribution to detect existence of outliers

Classification Report						
	Validation			Test		
	Precision	Recall	F1-score	Precision	Recall	F1-score
Benign	0.89	0.96	0.92	0.89	0.96	0.92
Brute Force	0.00	0.00	0.00	0.00	0.00	0.00
DoS Hulk	0.99	0.88	0.93	0.99	0.86	0.92
PortScan	0.84	0.89	0.86	0.84	0.89	0.86
accuracy			0.89			0.89
macro avg	0.68	0.68	0.68	0.68	0.68	0.68
weighted avg	0.85	0.89	0.87	0.85	0.89	0.87

Table 1: Validation and test classification reports of the best shallow model (hidden=64), showing similar performance across both sets, confirming generalization.

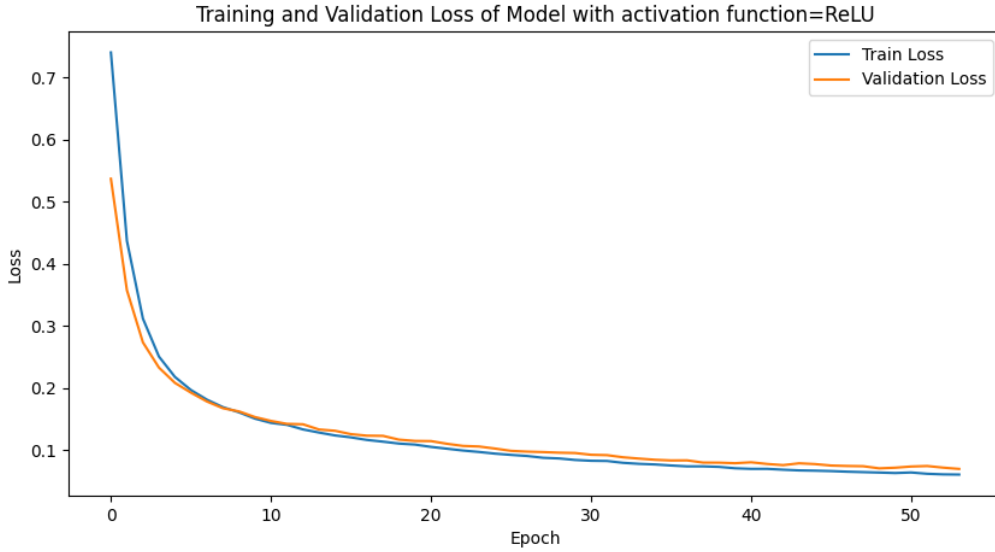


Figure 2: Training and Validation Curve of the model with ReLU activation function

2.2 ReLU activation

On Table 1 you can see that the model failed on class *Brute Force*. It is related with activation function, linear activation cannot solve non-linear relation. In this case, I decide to change linear activation function with **ReLU**, which introduce non-linearity.

As shown Fig. 2, the model converged smoother and stable. The *Brute Force* class misclassification issue also resolved by activation function. After ReLU applied, accuracy (**0.98**) and f1-score (**0.95**) improved. Especially, f1-score corresponding to *Brute Force* worked ($0.00 \rightarrow 0.87$).

Finally, it is better to use validation during model selection and tuning, and only at the end use the test set for final evaluation and see model true generalization.

2.3 Impact of Port Feature

Some classes strongly depend on specific features, which is far from real case, for example: all *Brute Force* attacks came from destination port number equal to **80**, however in reality *Brute Force* attack can process from any port. To proof and fix this specific case we experimented testing previously trained model with port number replaced test set (**80** $\rightarrow$ **8080**).

We tested the model with two test sets, one we took as baseline and on the second port was replaced. Table 2 demonstrated that after port replacement, all candidates previously assigned as *Brute Force*, predicted as *Benign* (normal). During training, the model learned to associate *Destination*

Classification Report						
	Baseline			Port (80 → 8080)		
	Precision	Recall	F1-score	Precision	Recall	F1-score
Benign	0.99	0.97	0.98	0.92	0.97	0.95
Brute Force	0.79	1.00	0.88	0.00	0.00	0.00
DoS Hulk	0.97	0.97	0.97	0.97	0.97	0.97
PortScan	0.98	0.99	0.98	0.98	0.99	0.98
accuracy			0.97			0.93
macro avg	0.93	0.98	0.95	0.72	0.73	0.73
weighted avg	0.98	0.97	0.98	0.89	0.93	0.91

Table 2: Baseline and Port replaced classification reports, showing the impact of *Destination Port* feature on *Brute Force* class

Port 80 with *Brute Force*, rather than learning true attack behavior. At the end, we decided to totally remove *Destination Port* feature from the original dataset and preprocessed again.

After removing the port feature and reprocessing, PortScan samples dropped dramatically from **4849** to **285** (94%). This is because *PortScan* attacks are distinguished by scanning different destination ports, once this feature is removed, most *PortScan* samples become identical to each other and are eliminated as duplicates. As shown in Fig. 3, after removing the port feature the dataset became highly imbalanced, with PortScan heavily underrepresented. Due to strong class imbalance, we used **F1-score** as our primary evaluation metric throughout the remaining experiments.

2.4 Impact of Loss Function

We extracted the best hidden dimension (`hid_dim=64`), and the best activation function (**ReLU**), and trained the model with updated hyperparameters and the dataset. As expected, the model completely failed to classify *PortScan*, with all metrics equal to **0.00**, which dragged the macro F1-score down to **0.68**. We computed balanced class weights using sklearn’s `compute_class_weight` on the training partition, and passed them to the **CrossEntropyLoss** function, as shown in Fig. 4. The *F1-score* corresponding to *PortScan* class improved from 0.0 to **0.34**, while macro *F1-score* of the model increased from 0.68 to **0.76**.

3 Deep Neural Network

In this section we extended our models by implementing more than a single hidden layer, creating Deep Feed Forward Neural Networks. Concretely, we selected the best hidden dimension architecture, optimal batch size, and optimizer with its optimal parameters.

3.1 Architectures

We trained six Deep FFNNs varying only the hidden dimensions: (8, 8, 8), (8, 16, 32), (8, 8, 8, 8), (32, 16, 16, 8), (8, 8, 8, 8, 8), (8, 16, 32, 16, 8), with fixed hyperparameters shown in Table 3. The best model was selected based on **minimum validation loss**. All six models have converged, on the models with static number of neurons per each hidden layer, train and validation losses smoothly decreased. While other models’ curve showed noisy curve. As shown in Fig. 5, the best model (8, 16, 32) achieved the lowest validation loss of less than **0.1284**, while the second the best architectures (8, 8, 8, 8) and (32, 16, 16, 8) performed similarly to each other with validation loss 0.13X. The remaining architectures showed higher losses. On the test set,

Hyperparameter	Value
Activation	ReLU
Weight Initialization	Default
Batch Size	64
Loss Function	Cross-Entropy
Optimizer	AdamW
Learning Rate	5e-4
Epochs & Early Stopping	50
Regularization	None

Table 3: Fixed hyperparameters for Deep FFNN experiments

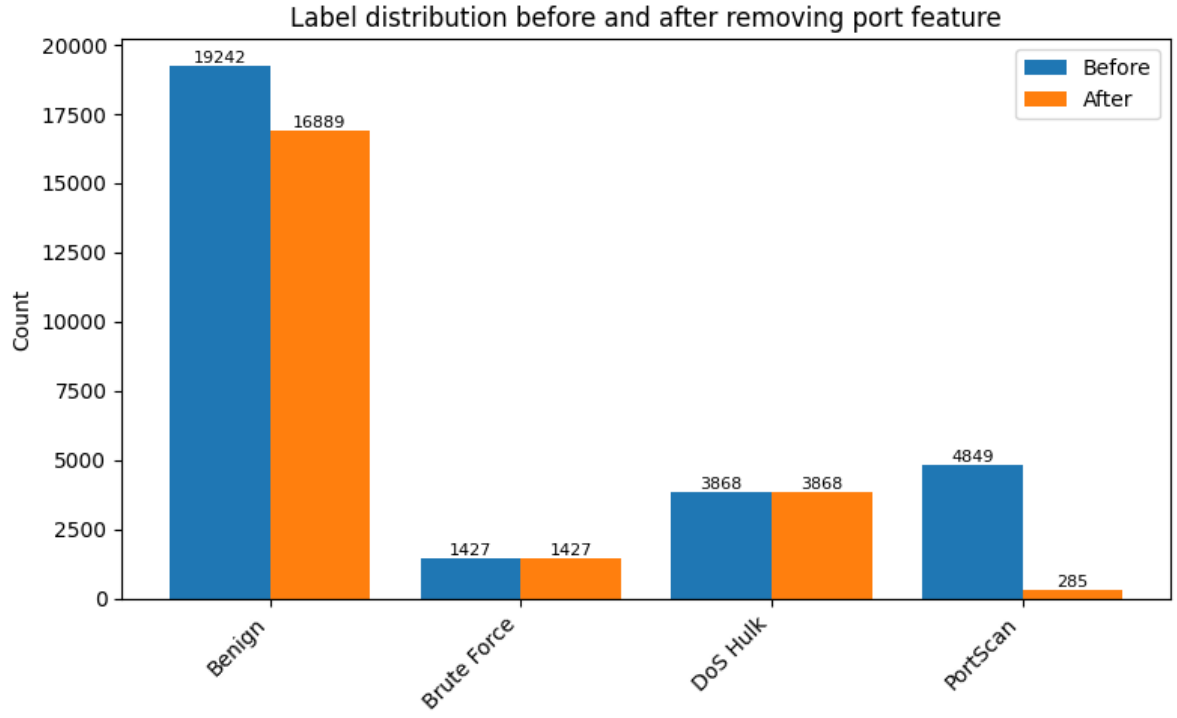


Figure 3: Label distribution, which shows only *Benign* and *PortScan* affected

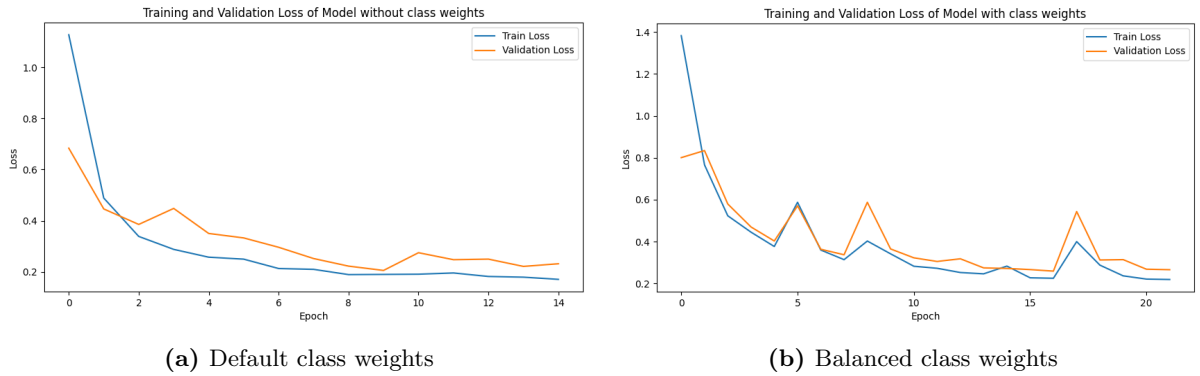


Figure 4: Loss curves without (left) and with (right) balanced class weights. The weighted loss introduces more noise but improved macro *F1-score*.

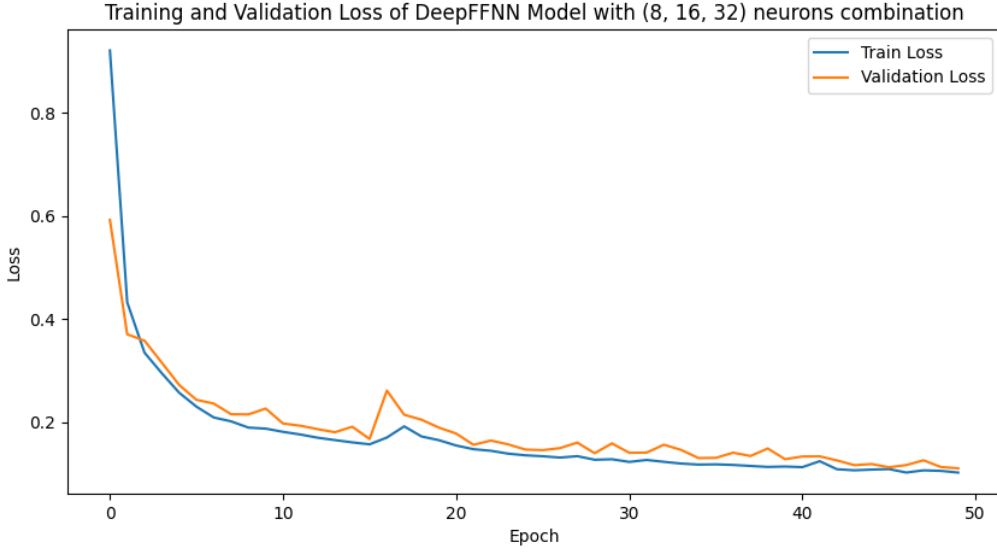


Figure 5: Training and validation curve of (8, 16, 32) model

the best model (8,16,32) achieved a macro *F1-score* of **0.88**. While the class-wise details shown on Fig. 6.

3.2 Batch Size

We experimented with batch sizes: 4, 64, 256, 1024. While all models have converged, their training behavior was different, larger batch sizes trained faster per epoch due to fewer gradient updates, while batch size 4 was significantly slower. The batch size with 4, was too noisy than other models, while batch sizes with larger numbers like 256 and 1024 suffered on classifying minority class. It started on `batch_size=256`, which shared very low metrics corresponding to *PortScan* class, at the same time `batch_size=1024` totally failed on this class with all metrics equal to 0.00.

Finally, the value for batch size selected as batch size equal to **64**, which has minimum validation loss value, at the same time having smoother curve comparing to `batch_size=4`.

3.3 Optimizer

The final tuning parameter was - optimizers, we started with Stochastic Gradient Descent, passed some momentum parameters on it: 0.1, 0.5, 0.9 and finally we applied state of the art: **AdamW**. The SGD without momentum was the slowest one, while adding momentum made it faster. However, still big risk of oscillation. The best option was as expected **AdamW** with **0.095** at epoch 50, which performed smoother and faster across all other models.

During training of different models, we understood another important hyperparameters. There are *Learning rate* and also *Number of epochs*, which is depended on early stopping.¹ We experimented four models with: 1e-4, 5e-4, 1e-3, 5e-3 learning rates. The models with too low and large learning rates failed on minority class (*PortScan*), because too low learning rate converges too slowly and may get stuck, while too high causes unstable training and overshooting, both harming minority class performance. The models with learning rate 1e-3 and 5e-4 showed better metrics, while by minimum validation loss won the model with learning rate equal to **1e-3**.

Hyperparameter	Value
Hidden Dimension	(8, 16, 32)
Activation	ReLU
Weight Initialization	Default
Batch Size	64
Loss Function	Cross-Entropy
Optimizer	AdamW
Learning Rate	1e-3
Epochs & Early Stopping	50
Regularization	None

Table 4: Final the best Deep Feed Forward Neural Network hyperparameters

¹*Early stopping* halts training when validation loss stops improving, controlled by `min_delta` and `patience`

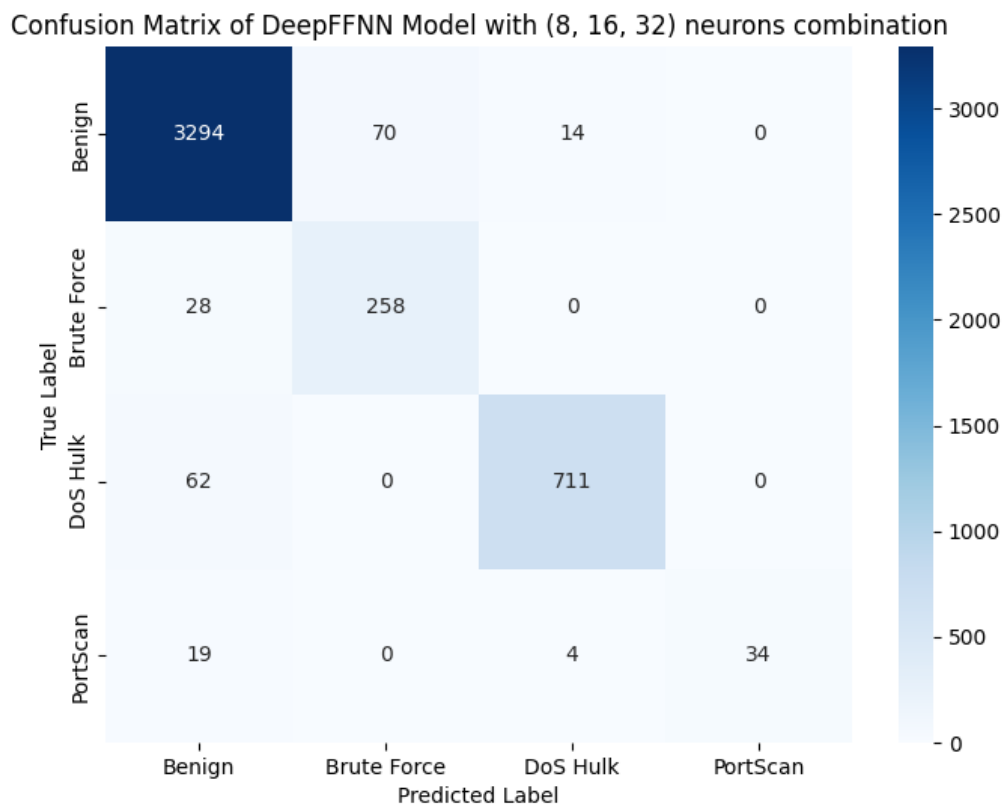


Figure 6: Confusion matrix of best model (8,16,32) on test set. *PortScan* shows lower recall due to class imbalance, with **19** out of 57 samples misclassified as *Benign*

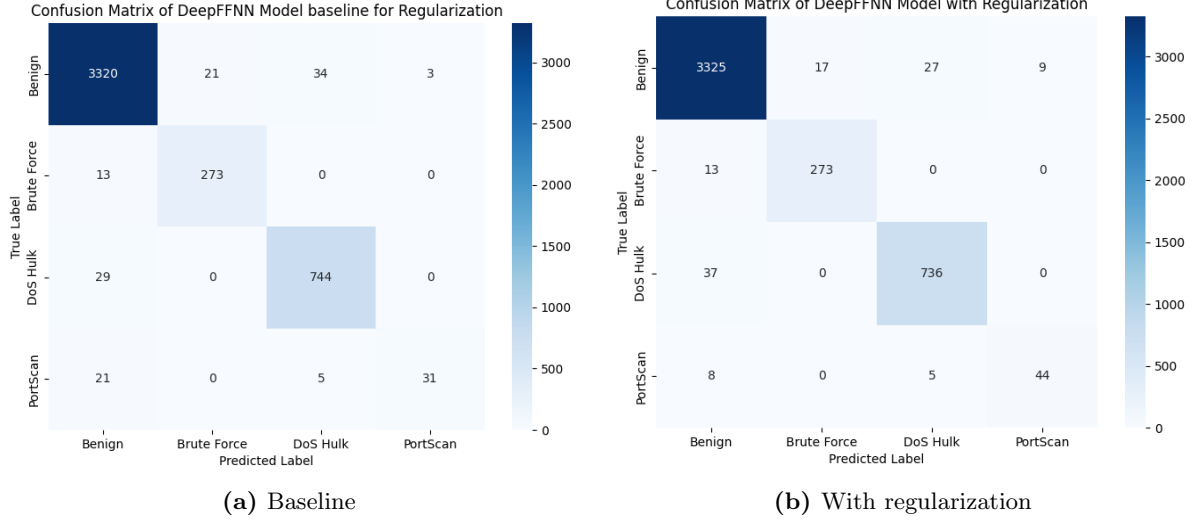


Figure 7: Confusion matrix without (left) and with (right) regularization.

Finally, we completed hyperparameter tuning, the selected parameters are summarized in Table 4. We reinitialized and trained the best model, after we tested it. As shown on Table 5, the results on both validation and test sets are similar, which proves the model generalization.

Classification Report						
	Validation			Test		
	Precision	Recall	F1-score	Precision	Recall	F1-score
Benign	0.98	0.99	0.98	0.98	0.99	0.98
Brute Force	0.94	0.93	0.93	0.94	0.94	0.94
DoS Hulk	0.96	0.94	0.95	0.97	0.94	0.95
PortScan	0.86	0.77	0.81	0.83	0.77	0.80
accuracy			0.97			0.97
macro avg	0.94	0.91	0.92	0.93	0.91	0.92
weighted avg	0.97	0.97	0.97	0.97	0.97	0.97

Table 5: Validation and test classification reports of the best Deep FFNN, showing similar performance across both sets, confirming generalization.

4 Overfitting and Regularization

In this section, we focused on regularization techniques. More specifically, we experimented *Weight decay*. First we trained and tested the model as baseline with given hyperparameters: six layers as 256, 128, 64, 32, 16, ReLU activation, 128 batch size, 5e-4 learning rate and 50 epochs, other parameters the same as previous experiments. The baseline model showed no significant overfitting, as validation and test performance remained consistent.

Next, we trained three models with different weight decay values: 0, 1e-4, 1e-3. The model with weight decay **1e-3** achieved the lowest validation loss and best macro *F1-score*, though differences between the three values were marginal. Finally, we compared the results of both baseline and the best weight decay model on Fig. 7. Since the baseline model already was well performing ($f1=0.89$), the regularized model did not improve performance significantly, it only helped on classifying minor class (*PortScan*), for example: recall of *PortScan* class was 0.54, while after regularization it improved to **0.77**. As the baseline model already showed strong performance, dropout and batch normalization were not applied, since weight decay alone was sufficient to achieve stable results.